

```
import pandas as pd

data_path = "Advertising.csv"

# Read the CSV data from the link

data_df = pd.read_csv(data_path, index_col=0)

# Print out first 5 samples from the DataFrame

data_df.head()
```

```
import matplotlib.pyplot as plt

import matplotlib as mpl

from sklearn import linear_model

fig = plt.figure(figsize=(15,4))

gs = mpl.gridspec.GridSpec(1,3)

# Plot of sales vs TV

ax = fig.add_subplot(gs[0])

ax.scatter(data_df["TV"], data_df["sales"], color="red", marker=".")

ax.set_xlabel("TV")

ax.set_ylabel("sales")

# Plot of sales vs radio

ax = fig.add_subplot(gs[1])
```

```

ax.scatter(data_df["radio"], data_df["sales"], color="green", marker=".")

ax.set_xlabel("radio")

ax.set_ylabel("sales")


# Plot of sales vs newspaper

ax = fig.add_subplot(gs[2])

ax.scatter(data_df["newspaper"], data_df["sales"], color="blue", marker=".")

ax.set_xlabel("newspaper")

ax.set_ylabel("sales")


plt.show()

```

```

import numpy as np
import matplotlib.pyplot as plt
import matplotlib as mpl
from sklearn import linear_model

fig = plt.figure(figsize=(15, 4))
gs = mpl.gridspec.GridSpec(1, 3)

def train_plot(data_df, feature, ax, col):
    X = data_df[feature].values
    Y = data_df['sales'].values

    cov_XY = np.cov(X, Y)[0][1]
    cov_XX = np.cov(X, Y)[0][0]

    m = cov_XY / cov_XX
    b = np.mean(Y) - m * np.mean(X)

    y_hat = m * X + b

    print(feature)
    print("Manual:", m, b)

```

```

reg = linear_model.LinearRegression()
reg.fit(X.reshape(-1,1), Y)

print("Sklearn:", reg.coef_, reg.intercept_)
print()

ax.scatter(data_df[feature], data_df["sales"], color=col, marker=".")
ax.plot(X, y_hat, color="yellow")
ax.set_xlabel(feature)
ax.set_ylabel("sales")
ax.set_title("$y$ = %.3f + %.3f$x$" % (b, m))

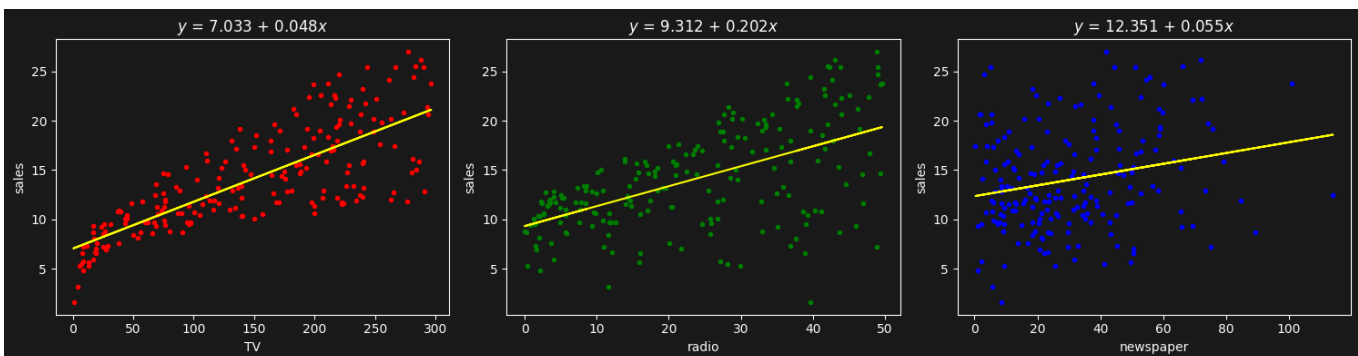
ax0 = fig.add_subplot(gs[0])
train_plot(data_df, "TV", ax0, "red")

ax1 = fig.add_subplot(gs[1])
train_plot(data_df, "radio", ax1, "green")

ax2 = fig.add_subplot(gs[2])
train_plot(data_df, "newspaper", ax2, "blue")

plt.tight_layout()
plt.show()

```



```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline

# Sample data
X = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10]).reshape(-1, 1)
y = np.array([3, 5, 7, 10, 15, 23, 35, 50, 70, 95])

# -----

```

```

# Linear Regression
# -----
linear_model = LinearRegression()
linear_model.fit(X, y)

linear_predictions = linear_model.predict(X)

# -----
# Polynomial Regression
# degree = 2 means quadratic curve
# -----
polynomial_model = make_pipeline(
    PolynomialFeatures(degree=2),
    LinearRegression()
)

polynomial_model.fit(X, y)

# Smooth X values for better curve drawing
X_smooth = np.linspace(1, 10, 200).reshape(-1, 1)
polynomial_predictions = polynomial_model.predict(X_smooth)

# -----
# Plot comparison
# -----
plt.scatter(X, y, label="Actual Data")

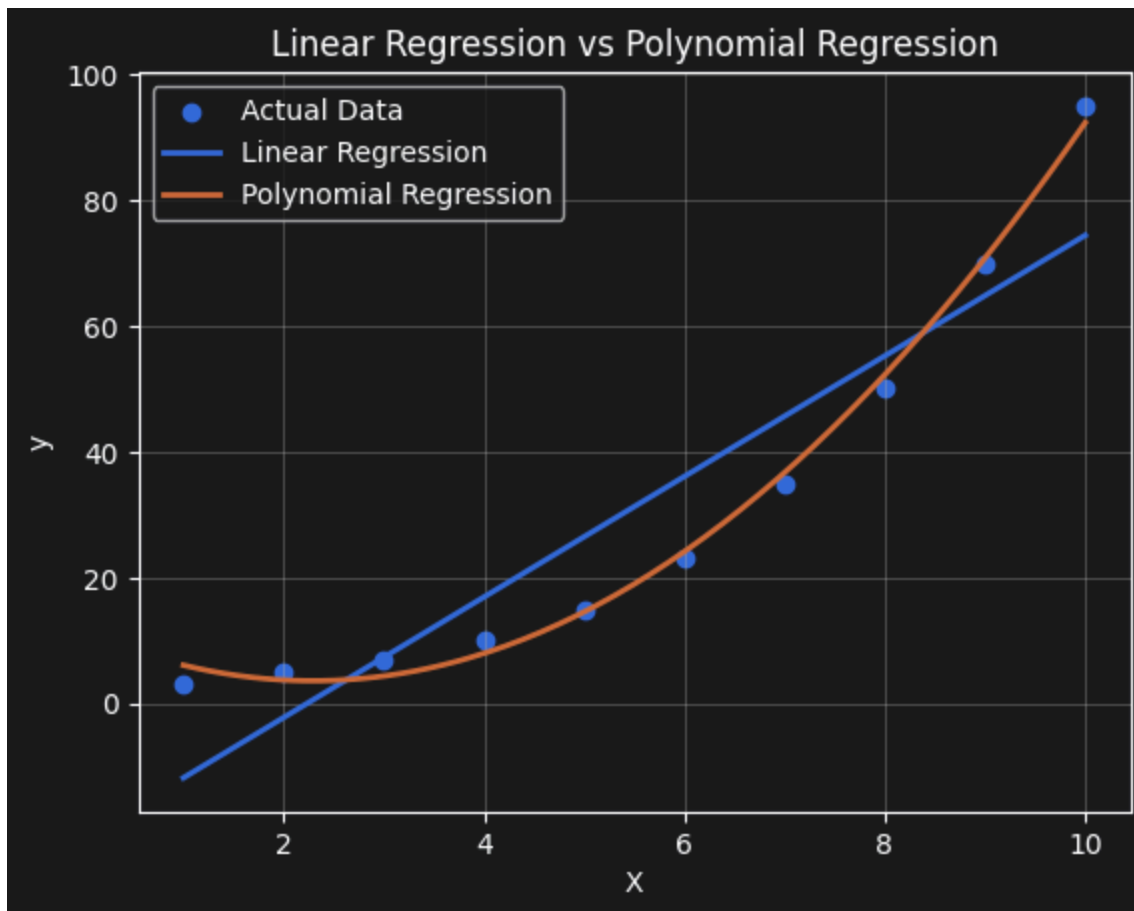
plt.plot(
    X,
    linear_predictions,
    label="Linear Regression",
    linewidth=2
)

plt.plot(
    X_smooth,
    polynomial_predictions,
    label="Polynomial Regression",
    linewidth=2
)

plt.xlabel("X")
plt.ylabel("y")
plt.title("Linear Regression vs Polynomial Regression")
plt.legend()

```

```
plt.grid(True)
plt.show()
```



```
# Linear regression equation
print("Linear intercept:", linear_model.intercept_)
print("Linear coefficient:", linear_model.coef_)

# Polynomial regression equation
poly_features = polynomial_model.named_steps["polynomialfeatures"]
poly_regression = polynomial_model.named_steps["linearregression"]

print("Polynomial intercept:", poly_regression.intercept_)
print("Polynomial coefficients:", poly_regression.coef_)
print("Feature names:", poly_features.get_feature_names_out(["x"]))
```

```
x_new = np.array([[15]])

linear_pred = linear_model.predict(x_new)
poly_pred = polynomial_model.predict(x_new)

print("Linear prediction:", linear_pred[0])
print("Polynomial prediction:", poly_pred[0])
```

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# -----
# Sample dataset
# x1 = study hours
# x2 = attendance percentage
# y = exam score
# -----
x1 = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
x2 = np.array([40, 45, 50, 55, 60, 65, 70, 75, 80, 85])

y = np.array([35, 40, 45, 50, 58, 65, 72, 78, 85, 92])

# Combine x1 and x2 into one input matrix
X = np.column_stack((x1, x2))

# -----
# Train multiple linear regression model
# -----
model = LinearRegression()
model.fit(X, y)

print("Intercept:", model.intercept_)
print("Coefficients:", model.coef_)

# -----
# Create grid for regression plane
# -----
x1_range = np.linspace(x1.min(), x1.max(), 20)
x2_range = np.linspace(x2.min(), x2.max(), 20)

x1_grid, x2_grid = np.meshgrid(x1_range, x2_range)

# Predict y values for every point on the grid
y_grid = model.predict(
    np.column_stack((x1_grid.ravel(), x2_grid.ravel()))
)

y_grid = y_grid.reshape(x1_grid.shape)

# -----
# 3D Plot
# -----
fig = plt.figure(figsize=(10, 7))

```

```

ax = fig.add_subplot(111, projection="3d")

# Actual data points
ax.scatter(x1, x2, y, label="Actual Data", s=50)

# Regression plane
ax.plot_surface(x1_grid, x2_grid, y_grid, alpha=0.5)

ax.set_xlabel("Study Hours")
ax.set_ylabel("Attendance Percentage")
ax.set_zlabel("Exam Score")

ax.set_title("3D Multiple Linear Regression")
ax.legend()

plt.show()

```

```

import numpy as np
import plotly.graph_objects as go
from sklearn.linear_model import LinearRegression

# -----
# Sample dataset
# -----
x1 = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
x2 = np.array([40, 45, 50, 55, 60, 65, 70, 75, 80, 85])

y = np.array([35, 40, 45, 50, 58, 65, 72, 78, 85, 92])

# Combine x1 and x2
X = np.column_stack((x1, x2))

# -----
# Train model
# -----
model = LinearRegression()
model.fit(X, y)

print("Intercept:", model.intercept_)
print("Coefficients:", model.coef_)

# -----
# Create grid for regression plane
# -----
x1_range = np.linspace(x1.min(), x1.max(), 20)

```

```

x2_range = np.linspace(x2.min(), x2.max(), 20)

x1_grid, x2_grid = np.meshgrid(x1_range, x2_range)

# Predict exam score for every grid point
y_grid = model.predict(
    np.column_stack((x1_grid.ravel(), x2_grid.ravel()))
)

y_grid = y_grid.reshape(x1_grid.shape)

# -----
# Plotly 3D Interactive Plot
# -----
fig = go.Figure()

# Regression plane
fig.add_trace(
    go.Surface(
        x=x1_grid,
        y=x2_grid,
        z=y_grid,
        opacity=0.6,
        name="Regression Plane"
    )
)

# Actual data points
fig.add_trace(
    go.Scatter3d(
        x=x1,
        y=x2,
        z=y,
        mode="markers",
        marker=dict(size=6),
        name="Actual Data"
    )
)

fig.update_layout(
    title="Interactive 3D Multiple Linear Regression",
    scene=dict(
        xaxis_title="Study Hours",
        yaxis_title="Attendance Percentage",
        zaxis_title="Exam Score"
    )
)

```



```
)
```

```
fig.show()
```